

Sistema de atendimento medico em python]

A) Quais são as entradas do sistema?

Nome do paciente.

Idade do paciente.

Código do tipo de atendimento (1 a 4).

- Confirmação de convênio (Sim/Não).
- Confirmação de cadastro de novo paciente (Sim/Não).

B) Quais são os processamentos?

- Verificação e aplicação da prioridade com base na idade.
- Cálculo do valor do atendimento conforme o código selecionado.
- Aplicação da regra de desconto para pacientes com convênio.
- Cálculo do valor final pago por cada paciente.
- Acumulação do faturamento bruto, faturamento líquido e total de descontos.
- Contagem do número total de pacientes, atendimentos prioritários e com convênio.
- Determinação do maior valor pago e do menor valor pago.
- Cálculo da média dos valores finais pagos.
- Condicional para classificação do movimento do dia (baixo, moderado ou alto).

C) Quais são as saídas?

Valor individual a ser pago por cada paciente.

Total de pacientes cadastrados.

Total de atendimentos prioritários.

Total de pacientes que utilizaram convênio.

Faturamento bruto total do dia.

Faturamento líquido total do dia.

Total acumulado de descontos concedidos.

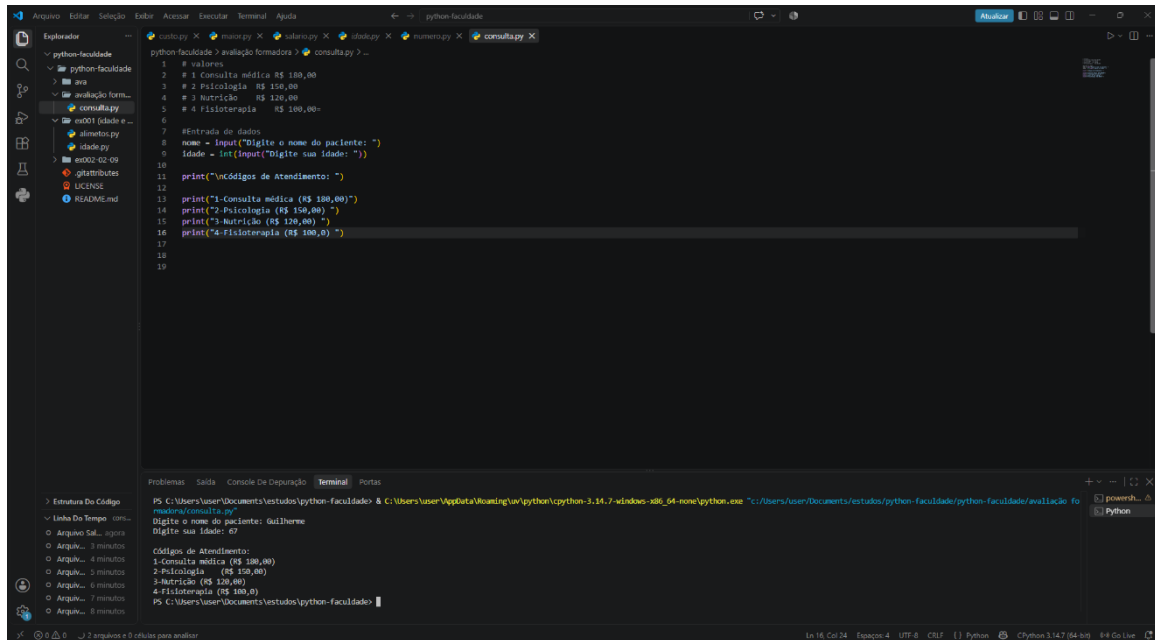
Classificação do movimento do dia.

Maior valor e menor valor pago. Média final do valor dos atendimentos.

D) Tabela de Variáveis:

Variável	Tipo	Finalidade
nome	Texto (str)	Nome do paciente
idade	Inteiro (int)	Idade
codigoAtendimento	Inteiro (int)	Tipo de atendimento
valorAtendimento	Real (float)	Valor original
convenio	Texto (str)	Indica se possui convênio
prioritario	Booleano (bool)	Indica prioridade
desconto	Real (float)	Valor do desconto
valorFinal	Real (float)	Valor final

-Testes realizados durante a execução do projeto:



Esta captura de tela foi tirada no início do projeto, momento em que a tabela de atendimentos estava sendo desenvolvida.

Nesta página, estão apresentados alguns testes em que os resultados não ocorreram da forma esperada, tornando necessárias correções na lógica e na estrutura do código

```
#repetição
totalpaciente = 0
totalatendimento = 0
totalconvenio = 0
fatbruto = 0
fatliquido = 0
totaldesc = 0
maior = 0
menor = 0
continuar = "s"
while continuar == "s":
```

```
if totalpaciente ==1:
    maior = custo
    menor = custo
else:
    if custo> maior:
        maior = custo
    if custo < menor:
        menor = custo
```

```

Digite o nome do paciente: aaaaaaaaaa
Digite sua idade: 11

Códigos de Atendimento:
1-Consulta médica (R$ 180,00)
2-Psicologia (R$ 150,00)
3-Nutrição (R$ 120,00)
4-Fisioterapia (R$ 100,0)
Digite o código de atendimento (1 a 4): 1
Possui convenio? (S/N):s

Voce devera pagar: 135.00

Deseja cadastrar outro paciente? (S/N):n
O total de pacientes foi: 1
O total de atendimentos prioritarios foi de: 1
O total de pacientes com convenio foi: 1
O faturamento bruto foi de: 180.00
O faturamento liquido foi de: 135.00
o total de descontos aplicados foram de: 45.00
Classificação do movimento: movimento baixo
O maior valor pago foi: 135.00
O menor valor pago foi: 0.00
A media foi: 135.00
PS C:\Users\user\Documents\estudos\python-faculdade>

```

```

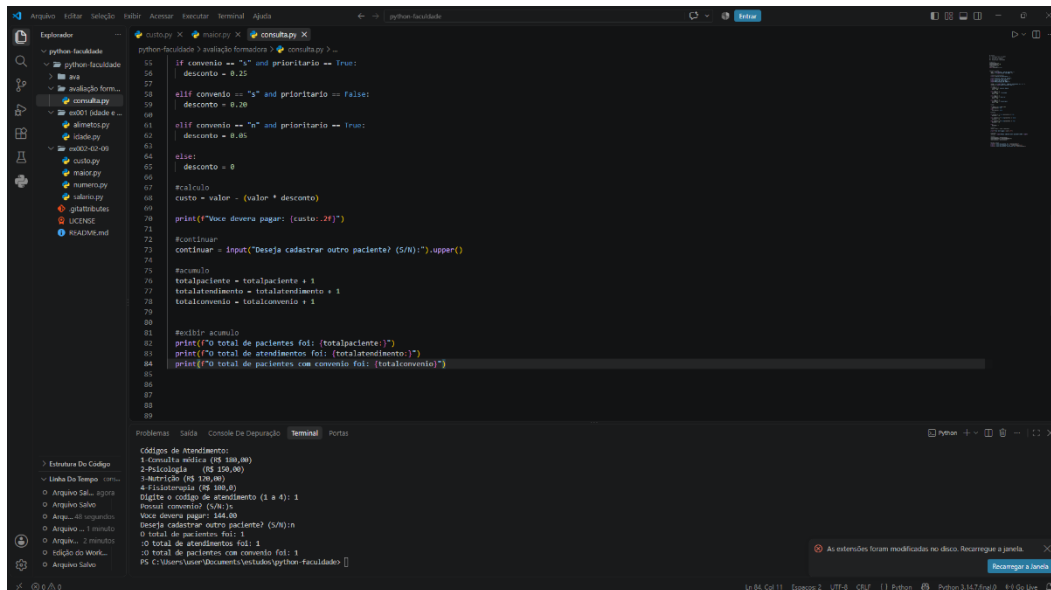
#repetição
totalpaciente = 0
totalatendimento = 0
totalconvenio = 0
fatbruto = 0
fatliquido = 0
totaldesc = 0
continuar = "s"
while continuar == "s":

```

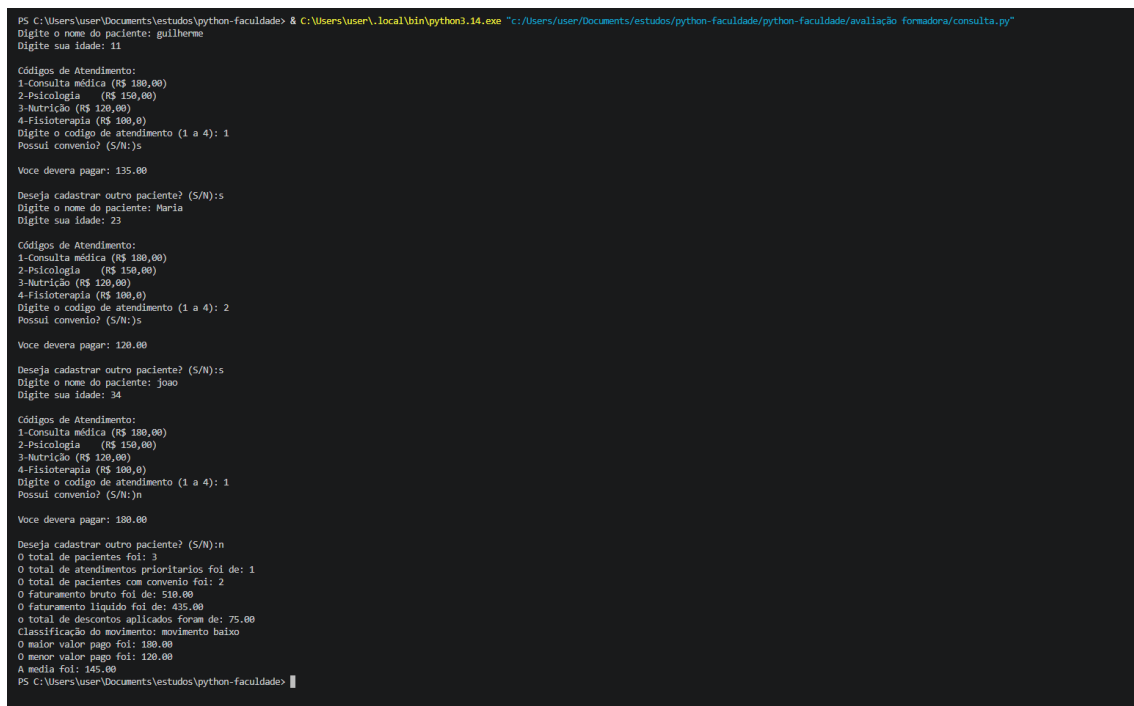
```

#exibir acumulo
print(f"O total de pacientes foi: {totalpaciente}")
print(f"O total de atendimentos prioritarios foi de: {totalatendimento}")
print(f"O total de pacientes com convenio foi: {totalconvenio}")
print(f"O faturamento bruto foi de: {fatbruto:.2f}")
print(f"o total de descontos aplicados foram de: {totaldesc:.2f}")

```



Esta captura de tela foi tirada com um avanço maior do projeto, quando a estrutura do código já estava quase finalizada.



Esta captura de tela demonstra o projeto finalizado, com todas as funcionalidades e cálculos executados com sucesso.

Perguntas para reflexão

1. Por que foi necessário utilizar variáveis diferentes para armazenar nome, idade e valor?

Porque cada informação possui uma natureza e tipo de dado distinto no computador: nomes são cadeias de caracteres (string), idades são números inteiros (int) sem casas decimais, e valores monetários são números com casas decimais (float). Cada tipo exige tratamento e operações matemáticas/lógicas específicas.

2. Qual a diferença entre uma variável inteira e uma variável real?

Variáveis inteiras (int) armazenam apenas números inteiros e exatos (sem vírgula/ponto decimal), sendo ideais para contagens como a idade ou quantidade de pessoas. Variáveis reais (float) armazenam números fracionários com ponto decimal, necessárias para medições e operações financeiras como preços e médias.

3. Em quais situações foram utilizadas estruturas condicionais?

Foram usadas no fluxo do programa para: testar a prioridade por idade, determinar o preço com base no código escolhido, verificar a concessão de desconto por convênio, redefinir as variáveis de maior e menor valor a cada iteração, e classificar o movimento do dia (baixo, moderado ou alto).

4. Por que uma estrutura de repetição é mais adequada que repetir manualmente o código para cada paciente?

Porque ela evita a duplicação desnecessária de código, reduz a probabilidade de erros e permite que o sistema processe qualquer quantidade de atendimentos de forma dinâmica, adaptando-se às entradas em tempo de execução.

5. Qual a diferença entre um contador e um acumulador?

Um contador é incrementado em um valor fixo (geralmente +1) para registrar a ocorrência de eventos (ex: totalpaciente += 1). Um acumulador soma valores variáveis a cada iteração para consolidar um montante total (ex: fatbruto += valor).

6. O que aconteceria se o usuário informasse um código de atendimento inexistente?

Na versão básica sem validação, o script não identificaria o preço ou aplicaria a condição else, resultando em cálculos incorretos ou em erro de execução por variável não definida.

7. Como o algoritmo poderia ser alterado para permitir o cadastro de milhares de pacientes?

Substituindo a estrutura baseada apenas em variáveis individuais por estruturas de dados como listas ou dicionários, ou ainda integrando o sistema a um banco de dados persistente.

8. Quais partes do algoritmo poderiam ser transformadas em funções?

A exibição do menu de serviços, o cálculo de descontos por convênio, a validação de dados de entrada e a rotina de emissão do relatório final.

9. Como você identificaria possíveis erros no algoritmo?

Realizando testes manuais com dados de borda (ex: valores negativos, zero, códigos fora da faixa de 1 a 4), aplicando chamadas de `print()` ao longo do código para rastrear os valores intermediários ou utilizando o depurador (*debugger*) da IDE.

10. Quais seriam os próximos recursos que poderiam ser acrescentados ao sistema?

Validação rigorosa dos dados informados, persistência dos dados cadastrados em arquivo ou banco de dados, geração de relatórios exportáveis e implementação de uma interface gráfica web ou desktop.